FASE II. EJECUCIÓN

Documento de validación y pruebas

Materia:	*Gestión de Proyectos de Inteligencia Artificial*
Actividad:	*Fase II*
Alumno(a):	*Paola Quinto Martínez*
Matrícula:	*03039650*
Docente:	*Luis Ariel Vázquez Piña*
Fecha de entrega:	*02 Noviembre 2026*

FASE II. EJECUCIÓN

Herramienta de IA utilizada	¿La usaste?	Propósito / Descripción de uso
ChatGPT / OpenAI	☒ Sí ☐ No	Apoyo para organizar el contenido, revisar la redacción y preparar explicaciones. La información se contrastó con los resultados de la ejecución local.
Claude (Anthropic)	☐ Sí ☒ No	No se utilizó.
Gemini (Google)	☐ Sí ☒ No	No se utilizó.
Copilot (Microsoft)	☐ Sí ☒ No	No se utilizó.
Codex (OpenAI)	☒ Sí ☐ No	Apoyo en la creación y revisión de archivos de la aplicación, pruebas y documentación. El funcionamiento se comprobó en el entorno local.

FASE II. EJECUCIÓN Documento de validación y pruebas

1. Introducción

En esta actividad se preparó una aplicación que permite escribir un comentario y obtener una clasificación de sentimiento. La solución se basa en un modelo previamente utilizado para comparar resultados de análisis de sentimientos. El objetivo de esta fase fue integrar la aplicación, ejecutarla en Docker y comprobar su funcionamiento local. La validación se enfocó en revisar que sus componentes se comunicaran correctamente y que la aplicación mostrara un resultado al ingresar un texto.

2. Desarrollo

2.1 Aplicación y componentes

La solución se organizó en dos partes: una interfaz para escribir comentarios y consultar el resultado, y un servicio que recibe el texto y solicita la clasificación al modelo. La comunicación entre ambas partes se realiza mediante una API. La aplicación se preparó con Docker Compose para iniciar los componentes de manera conjunta.

2.2 Pruebas y resultados

Las pruebas automatizadas reportaron 13 casos aprobados. Se revisaron, entre otros, el estado del servicio, el envío de texto, campos vacíos, datos incorrectos, límites de longitud y respuestas ante errores. Estas pruebas utilizaron respuestas controladas para comprobar el comportamiento de la aplicación; no todas ejecutaron el modelo real.

2.3 Evidencia de ejecución local

La interfaz se abrió en http://localhost:8501 y la documentación de la API en http://localhost:8000/docs. En las capturas anexas se muestran distintos comentarios y sus resultados. La clasificación puede variar según el texto y las limitaciones del modelo, que está orientado principalmente a mensajes en inglés.

Evidencia visual de ejecución local

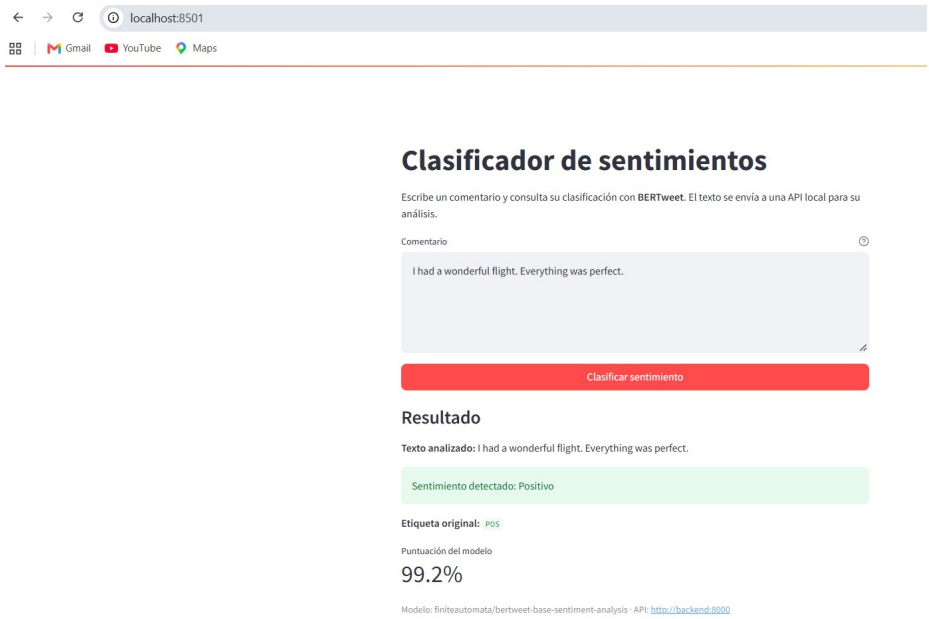


Figura 1. Clasificación de un comentario con sentimiento positivo.

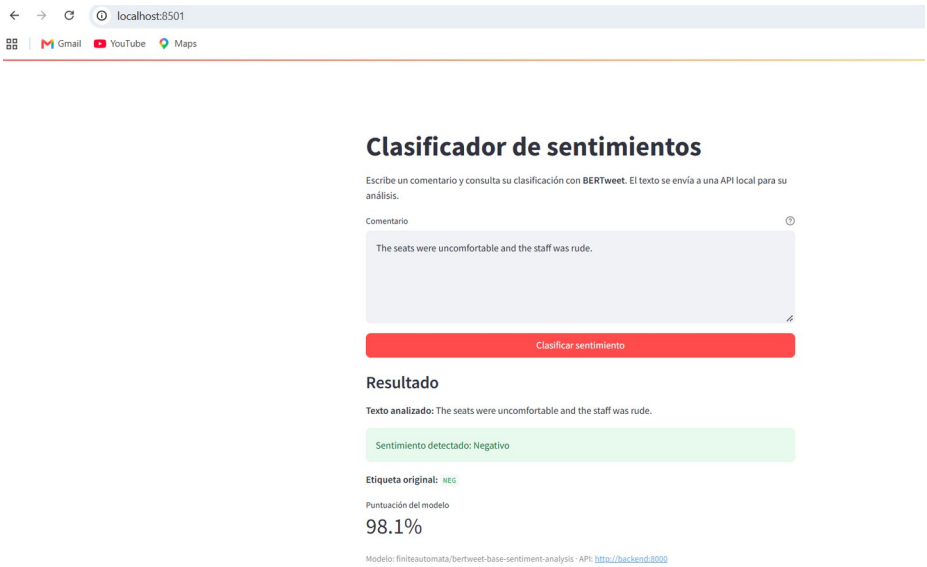


Figura 2. Clasificación de un comentario con sentimiento negativo.

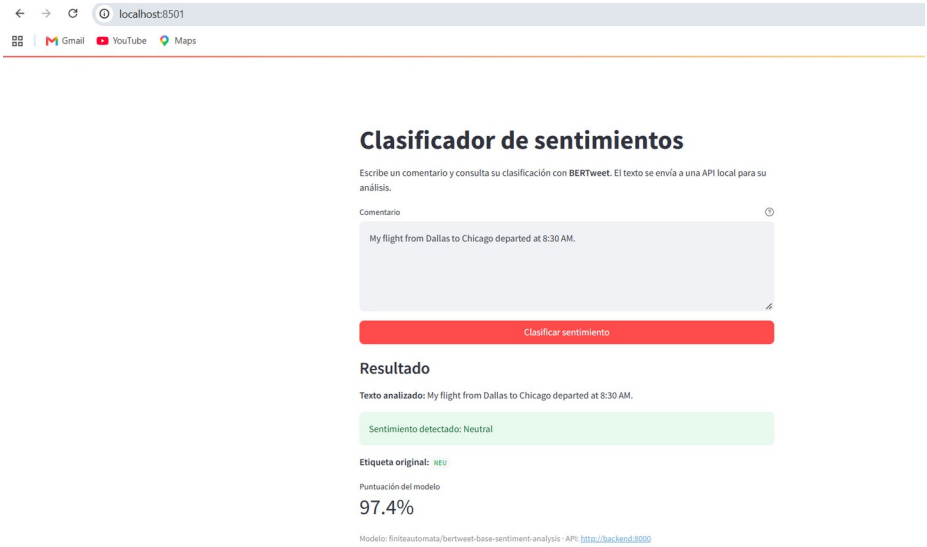


Figura 3. Clasificación de un comentario con sentimiento neutral.

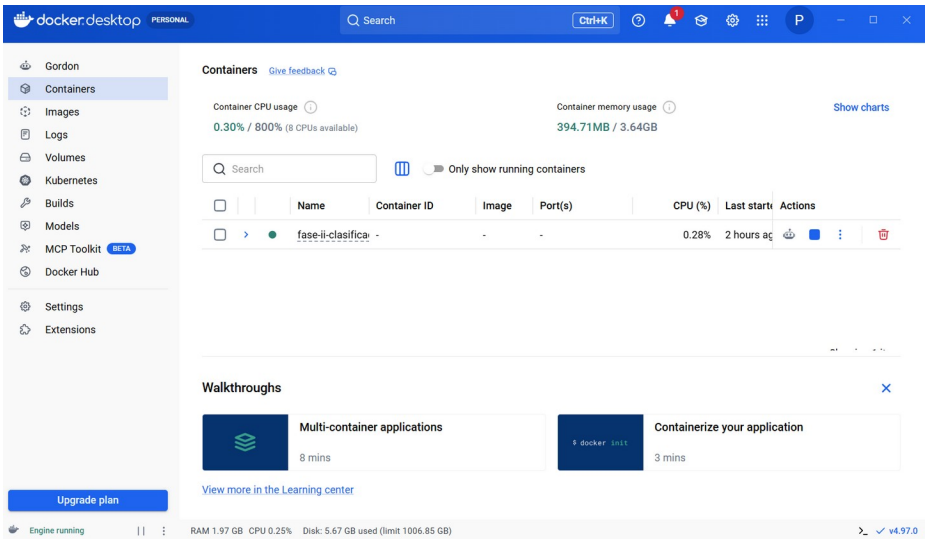


Figura 4. Contenedor de la aplicación visible en Docker Desktop.



Figura 5. Documentación de la API de análisis de sentimientos en FastAPI.

2.4 Pruebas realizadas y pendientes

Pruebas realizadas:
• 13 pruebas automatizadas aprobadas (pytest), con respuestas simuladas para validar contratos y manejo de errores.
• Inicio de los servicios con Docker Compose y comprobación del estado del backend.
• Comunicación entre Streamlit y FastAPI, con una predicción real visible en la interfaz.
• 23 casos extremos enviados mediante solicitudes HTTP reales a la API: idiomas, emojis, sarcasmo, caracteres especiales, texto largo y entradas inválidas. Los 23 códigos HTTP coincidieron con los esperados.

Pruebas pendientes:
• Evaluación del desempeño con un conjunto independiente de comentarios etiquetados y métricas como precisión, recall y F1.
• Instrumentación específica para medir la longitud de la secuencia después del truncamiento a 128 tokens; el caso C15 comprobó la aceptación HTTP, no la longitud tokenizada.
• Prueba controlada de indisponibilidad de Hugging Face y de interrupciones o demoras de la API.
• Despliegue real en un proveedor de nube; esta fase documenta una estrategia, no una publicación en producción.

2.5 Entorno y pruebas automatizadas

Entorno de pruebas

Ejecución observada el 2026-11-02 en Python 3.13.3 con pytest 8.3.4, FastAPI 0.115.6, httpx 0.28.1 y Pydantic 2.14.2. Las pruebas de API usan un analizador falso y no descargan pesos de Hugging Face. La prueba del cliente HTTP simula la respuesta del backend. Docker CLI 29.9.2 y Docker Compose 5.6.1 están instalados. docker compose config validó correctamente el archivo. La imagen de ejecución propuesta usa Python 3.11.

Casos de prueba automatizados

ID	Escenario / entrada	Resultado esperado	Resultado obtenido	Estado
API-01	GET /health después del inicio	HTTP 200 e indicador model_loaded: true	Pasó con analizador inyectado	Aprobada (simulada)
API-02	Comentario no vacío	HTTP 200, texto, sentimiento, etiqueta original y puntuación entre 0 y 1	Pasó con respuesta simulada	Aprobada (simulada)
API-03	Cadena vacía	HTTP 422 con JSON de error de validación	Pasó	Aprobada (simulada)
API-04	Campo text ausente	HTTP 422 que indica el campo requerido	Pasó	Aprobada (simulada)
API-05	Valor de text que no es cadena	HTTP 422 con mensaje específico	Pasó	Aprobada (simulada)
API-06	Solo espacios y saltos de línea	HTTP 422 con mensaje claro	Pasó	Aprobada (simulada)
API-07	Más de 5,000 caracteres	HTTP 422 con mensaje de límite	Pasó	Aprobada (simulada)
API-08	Exactamente 5,000 caracteres	Solicitud aceptada	Pasó con respuesta simulada	Aprobada (simulada)
API-09	Error del analizador	HTTP 503 con mensaje general y sin detalles internos	Pasó con excepción simulada	Aprobada (simulada)
WEB-01	Cliente frontend envía texto	POST al endpoint /predict, valida estado y decodifica JSON	Pasó con cliente HTTP simulado	Aprobada (simulada)
LABEL-01	NEG, NEU, POS	Mapeo a Negativo, Neutral y Positivo	Pasó en prueba unitaria	Aprobada (unitaria)
MODEL-01	Secuencia mayor a 128 tokens	Tokenizador original usa truncamiento a 128 tokens	Texto largo C15 respondió 200; no se inspeccionó la longitud tokenizada.	Pendiente: falta medir directamente tokens tras truncamiento
DOCKER-01	Inicio con docker compose up --build -d	Servicios inician; interfaz y API responden localmente	Pasó: backend healthy y frontend running	Aprobada (Docker real, documentada)
MODEL-02	Inicio del contenedor y carga del checkpoint Hugging Face	Backend informa modelo cargado	Pasó: GET /health respondió model_loaded: true	Aprobada (Docker real, documentada)
INFER-01	POST real en /predict	Respuesta del checkpoint con texto, sentimiento, etiqueta y puntuación	Pasó desde el contenedor frontend: HTTP 200, sentimiento Positivo, etiqueta POS, puntuación 0.9918472766876221	Aprobada (inferencia real, documentada)
NET-01	Puerto del frontend publicado en Compose	127.0.0.1:8501 apunta al puerto 8501 del contenedor	Pasó tras añadir la red bridge frontend-ingress; docker compose ps muestra 127.0.0.1:8501->8501/tcp	Aprobada (Docker real, documentada)
WEB-02	Uso real de Streamlit contra FastAPI	Interfaz muestra el resultado de la predicción	Pasó en navegador: mostró el texto, Positivo, POS y 99.2%	Aprobada (Streamlit real, documentada)

Resultados obtenidos

Comando ejecutado: .venv\\Scripts\\python.exe -m pytest -q -p no:cacheprovider con el entorno virtual activo.

Resultado automatizado observado el 2026-11-02: 13 passed, 1 warning. El aviso es una DeprecationWarning de Starlette por el alias anyio.abc.BlockingPortal. Las pruebas

cubren salud, respuesta válida simulada, validaciones de texto, límites, mapeo de etiquetas, excepción simulada del analizador y llamada del cliente HTTP Streamlit.

La ejecución real de Compose construyó las imágenes y arrancó los dos servicios. El backend informó model_loaded: true. Se probó desde el contenedor frontend la conectividad a http://backend:8000/health (HTTP 200) y se envió una predicción real a /predict (HTTP 200; Positivo, POS, puntuación 0.9918472766876221). Inicialmente Streamlit no era accesible desde el host: el puerto solicitado no aparecía publicado. Se añadió frontend-ingress para conectar el frontend a una red bridge además de app-internal; se recreó únicamente el frontend. Después docker compose ps reportó 127.0.0.1:8501->8501/tcp, http://localhost:8501 respondió HTTP 200 y una clasificación iniciada en el navegador mostró Positivo, POS y 99.2%. El backend y el volumen de caché no se recrearon ni eliminaron durante el ajuste de red.

2.6 Pruebas extremas de la API

Metodología

Se ejecutó scripts/pruebas_extremas.py desde el entorno local contra http://127.0.0.1:8000, el puerto publicado por el backend ya activo en Docker Compose. Antes de los casos se comprobó docker compose ps: backend healthy y frontend activo; además, GET /health respondió status: ok y model_loaded: true. Cada caso realizó una solicitud HTTP real a POST /predict; no se usaron mocks ni se reconstruyeron contenedores.

El script ejecuta 23 casos contra la API real. El criterio de aprobación es que el código HTTP obtenido coincida con el esperado. Las salidas del sentimiento no se compararon con etiquetas de referencia, por lo que estos resultados no miden exactitud ni precisión. Las entradas extensas se abrevian en la tabla por legibilidad; la tabla muestra los resultados registrados para los 23 casos.

Resultados reales

Se ejecutaron 23 casos contra la API real en Docker: 23 códigos HTTP coincidieron con lo esperado y no hubo discrepancias. Las solicitudes válidas devolvieron HTTP 200; las solicitudes inválidas devolvieron HTTP 422. La tabla completa que sigue contiene la entrada, el código esperado y obtenido y, cuando hubo inferencia, el sentimiento, etiqueta original y puntuación del modelo. Las etiquetas no se contrastaron con un conjunto de referencia.

ID	Caso	Entrada	HTTP esperado	HTTP obtenido	Sentimiento	Etiqueta original	Puntuación
C01	Inglés positivo	The crew was helpful and kind.	200	200	Positivo	POS	98.8%
C02	Inglés negativo	My luggage was lost and nobody helped me… (41 car.)	200	200	Negativo	NEG	97.3%
C03	Inglés neutral	The flight departs at 9:30 from gate 12.	200	200	Neutral	NEU	96.6%
C04	Mixto	The flight was delayed, but the crew was… (54 car.)	200	200	Positivo	POS	85.8%
C05	Español positivo	El servicio fue excelente y el personal …	200	200	Positivo	POS	98.2%

		(51 car.)					
C06	Español negativo	Pésimo servicio, perdieron mi equipaje y… (53 car.)	200	200	Neutral	NEU	95.7%
C07	Español neutral	El vuelo sale a las 9:30 de la puerta 12… (41 car.)	200	200	Neutral	NEU	97.3%
C08	Sarcasmo	Oh great, another two hour delay. Just w… (53 car.)	200	200	Negativo	NEG	96.9%
C09	Emojis positivos	Amazing flight 😍✈️👏	200	200	Positivo	POS	98.9%
C10	Emojis negativos	Worst airline ever 😡👎	200	200	Negativo	NEG	98.2%
C11	Solo emojis	😀😀😀	200	200	Neutral	NEU	90.1%
C12	Caracteres especiales	<script>alert(1)</script> & "quotes" 'x'… (46 car.)	200	200	Neutral	NEU	96.2%
C13	Unicode no latino	飞行很棒，机组人员很友好。	200	200	Neutral	NEU	93.4%
C14	Un solo carácter	a	200	200	Neutral	NEU	86.6%
C15	Texto largo (>128 tokens)	The flight was great and the crew was ki… (1760 car.)	200	200	Positivo	POS	98.8%
C16	Límite exacto 5,000 caracteres	xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx… (5000 car.)	200	200	Neutral	NEU	93.1%
C17	5,001 caracteres	xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx… (5001 car.)	422	422	—	—	—
C18	Cadena vacía		422	422	—	—	—
C19	Solo espacios y saltos de línea	\n	422	422	—	—	—
C20	Campo text ausente	(sin campo)	422	422	—	—	—
C21	text no es cadena	123	422	422	—	—	—
C22	JSON inválido	{esto no es json	422	422	—	—	—
C23	Cuerpo de solicitud vacío		422	422	—	—	—

2.7 Análisis de casos relevantes

Análisis de casos relevantes

• C11, solo emojis positivos: el modelo devolvió Neutral / NEU (90.1 %). Se registra como comportamiento observado; no se había definido una etiqueta de referencia para emojis aislados.

• C08, sarcasmo: devolvió Negativo / NEG (96.9 %). Un solo ejemplo no permite concluir que el modelo detecte sarcasmo de forma general.

• C15, texto largo de 1,760 caracteres: fue aceptado y clasificado con HTTP 200. El proyecto configura truncamiento a 128 tokens; el caso no inspecciona directamente los tensores ni demuestra la calidad sobre las partes truncadas.

• C16, exactamente 5,000 caracteres: HTTP 200. C17, 5,001 caracteres: HTTP 422 con mensaje de límite. El límite de caracteres se comportó como está definido.

• C12 y C13, caracteres especiales y texto chino: HTTP 200. Se informa únicamente la salida del clasificador, no una corrección semántica esperada.

2.8 Evidencias disponibles y limitaciones

Evidencias disponibles

El documento incorpora capturas de Streamlit con resultados positivo, negativo y neutral (páginas 4 y 5), una evidencia de Docker Desktop y la documentación interactiva de FastAPI. Estas imágenes respaldan la ejecución local y la integración visual de los componentes.

Las pruebas HTTP verifican el código de respuesta esperado, no la exactitud semántica de las predicciones. No se calcularon métricas de desempeño con un conjunto de referencia etiquetado. Las puntuaciones del modelo no deben interpretarse como confianza calibrada.

3. Conclusiones

La ejecución local permitió comprobar la construcción y el inicio de los contenedores, la carga del modelo, la comunicación entre Streamlit y FastAPI y la generación de predicciones mediante el checkpoint real. Además, los 23 casos extremos obtuvieron el código HTTP esperado, incluidos los escenarios válidos e inválidos. Se observaron respuestas del modelo que pueden ser semánticamente inesperadas en español y ante emojis aislados; esto evidencia que una respuesta HTTP exitosa no garantiza una clasificación correcta. Las pruebas automatizadas con mocks validan contratos y manejo de errores, mientras que las pruebas extremas verifican respuestas HTTP y registran las inferencias sin compararlas con etiquetas de referencia. Por ello, no se puede afirmar una precisión general ni una capacidad de generalización. Como trabajo pendiente se propone evaluar un conjunto independiente etiquetado, medir directamente el truncamiento a 128 tokens y probar fallas de descarga y comunicación. El despliegue en nube permanece como propuesta documentada, no como servicio publicado.

Referencias Bibliográficas

- Hugging Face. (s. f.). finiteautomata/bertweet-base-sentiment-analysis. https://huggingface.co/finiteautomata/bertweet-base-sentiment-analysis
- Instituto Tecnológico y de Estudios Superiores de Monterrey. (s. f.). Tema 15. Despliegue local [Material didáctico de clase].
- Instituto Tecnológico y de Estudios Superiores de Monterrey. (s. f.). Tema 16. Despliegue en la nube [Material didáctico de clase].
- Docker. (s. f.). Docker Compose. https://docs.docker.com/compose/